REVEAL — Zero-Cost Deployment Plan

*A step-by-step roadmap to put REVEAL live on the public internet for **$0/month**, using only the free tiers of well-known cloud providers. Written to be followed top to bottom.*

> **Companion doc:** `ARCHITECTURE.md` explains how the app is built. This doc explains how to ship it.

---

1. The goal

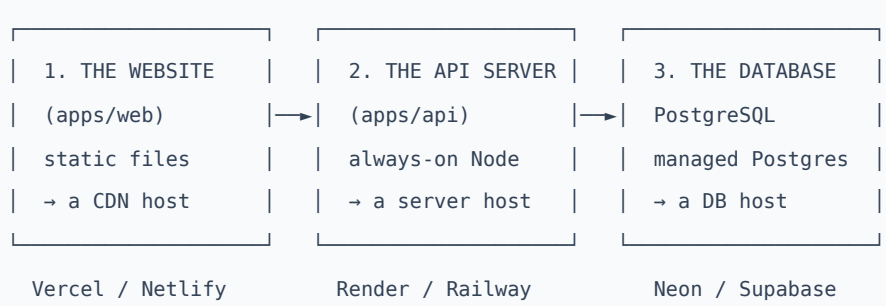
Get a working, shareable REVEAL — the real product, not a demo — running on the internet without a credit card bill. This is exactly what a pilot cohort or an investor demo needs: a live URL, real accounts, real reports.

We are **not** optimising for scale here. We're optimising for **$0 and simple**. Section 8 covers what to upgrade first when the pilot grows.

---

2. What we're actually deploying

REVEAL is three deployable pieces. Each maps to a different kind of free host:



Piece	What it is	What kind of host it needs
Website (`apps/web`)	Static files (built by Vite)	A static/CDN host — the easiest and cheapest thing to host
API (`apps/api`)	A long-running Node server	A server host that keeps a process alive and answers requests
Database	Managed PostgreSQL	A hosted Postgres provider

The good news: **the repo already ships deploy configs for the recommended path** — `vercel.json` (website) and `render.yaml` (API + database). You are wiring up something that was designed to be deployed, not inventing it.

---

3. Choosing the providers

3a. Hosting the website (static)

Provider	Free tier headline	Verdict for REVEAL
Vercel ✅	Generous static hosting, global CDN, automatic SPA rewrites, Git-push deploys	**Recommended.** `vercel.json` is already written for it.
Netlify	Very similar; 100 GB bandwidth/mo, 300 build min/mo	Excellent fallback; same shape of config
Cloudflare Pages	Unlimited bandwidth, generous builds	Great if bandwidth ever becomes the concern

All three serve our built files (`apps/web/dist`) essentially for free. We pick **Vercel** because the config already exists and its single-page-app routing ("send every path to `index.html` ") is pre-configured in `vercel.json` .

3b. Hosting the API server (always-on Node)

Provider	Free tier headline	The catch
Render ✅	Free web service, `render.yaml` already written, health checks	Free service **sleeps after ~15 min idle**; first request then takes ~30–60s to wake
Railway	Smooth DX	Free plan is trial credit, not a standing free tier — not truly $0 long-term
Fly.io	Real always-on small VMs	More setup (Docker); overkill for a pilot

We pick **Render** because `render.yaml` already defines the API service (build command, start command, health check, env vars) *and* a free Postgres database in one file. The one thing to know and plan around is the **cold start** (see §6).

3c. Hosting the database (PostgreSQL)

Provider	Free tier headline	The catch
Neon ✅	Serverless Postgres, ~0.5 GB storage, generous compute, branching	Compute **auto-suspends when idle**, then wakes in ~1s on first query
Supabase	~0.5 GB DB + storage + auth extras	Projects **pause after ~1 week of inactivity** on free tier; needs a manual/scheduled ping to stay warm
Render Postgres	Bundled in `render.yaml` , free plan	Free DB instances **expire after ~90 days** — fine for a pilot, plan a migration after

Two clean, fully-free routes:

- **Route A — All-Render (simplest):** use `render.yaml` as-is. API + Postgres come up together. Best for "I want it live tonight." Watch the ~90-day DB expiry.

- **Route B — Render API + Neon DB (most durable):** host the API on Render but point `DATABASE_URL` at a **Neon** database instead of Render's. No 90-day clock; Neon just sleeps and wakes. **Recommended for anything beyond a quick demo.** REVEAL's future storage need (`STORAGE_BUCKET` for portfolio/resume uploads) also pairs naturally with **Supabase Storage** if you go that way.

> Our database connection code already sets SSL for production and keeps a small connection pool (max 10) — sized deliberately for exactly these free tiers.

---

4. Recommended stack (the decision)

Layer	Provider	Config file already in repo
Website	**Vercel**	`vercel.json`
API server	**Render** (free web service)	`render.yaml`
Database	**Neon** (Route B) *or* Render Postgres (Route A)	`render.yaml` (Route A)

The milestones in §7 follow **Route B** (Render API + Neon DB) because it's the most durable free setup, and note where Route A is simpler.

---

5. Before you start — the prerequisites

- ☐ The code is pushed to GitHub (the repo these files live in).
- ☐ A **Vercel** account (sign in with GitHub).
- ☐ A **Render** account (sign in with GitHub).
- ☐ A **Neon** account (Route B) — or skip if using Route A.
- ☐ Locally: Node ≥ 20 and `pnpm` installed, so you can run the one-time database setup commands.
- ☐ A long random string ready to use as `JWT_SECRET` (e.g. run `openssl rand -base64 32`).
- ☐ *(Optional)* An Anthropic API key, **only** if you want live AI-written reports. Without it, REVEAL uses its built-in deterministic writer and works completely.

---

6. Free-tier limits to design around

These are the real constraints of a $0 deployment. None are blockers for a pilot; all are worth knowing before you demo.

Limit	Where	What it means in practice	How we handle it
API cold start	Render free service sleeps after ~15 min idle	First request after idle takes ~30–60s	Warm it before a demo (open the site a minute early), or add a free uptime pinger hitting `/health`
Database sleep	Neon auto-suspend / Supabase pause	First query after idle adds ~1s (Neon) or needs a wake (Supabase)	Neon wakes automatically; for Supabase, a scheduled ping keeps it warm
Render DB expiry	Render free Postgres	Instance is removed after ~90 days	Use Neon (Route B), or migrate before the deadline
Bandwidth	Vercel/Netlify (~100 GB/mo)	Thousands of report views before it matters	Non-issue at pilot scale
Build minutes	Vercel/Netlify/Render	Each deploy consumes a few minutes	Non-issue unless deploying dozens of times a day
DB storage	Neon/Supabase (~0.5 GB)	Tens of thousands of text records fit easily	Store large files (images/resumes) in object storage, not the DB
Compute hours	Render free web service (monthly cap)	Plenty for one always-sleeping pilot service	Non-issue at pilot scale

> **The single most important caveat:** a free API server *sleeps*. The very first visitor after a quiet period waits ~30–60 seconds. This is normal and expected on free tiers — just warm it up before any live demo.

---

7. The launch, milestone by milestone

Milestone 1 — Provision the database

Route B (Neon, recommended):

1. Create a new project in Neon; it gives you a **connection string** that looks like `postgresql://user:password@ep-xxx.neon.tech/reveal?sslmode=require`.
2. Copy that string somewhere safe — it becomes `DATABASE_URL` everywhere.

Route A (Render Postgres): skip this step; `render.yaml` creates the database for you in Milestone 3, and Render supplies its `DATABASE_URL`.

Milestone 2 — Set up the database schema (one time, from your laptop)

Point your local machine at the *production* database and load the schema + the starting content:

```
# Use the production connection string just for these two commands:
export DATABASE_URL="postgresql://…your-neon-or-render-url…"

pnpm install
pnpm --filter @reveal/shared build
pnpm --filter @reveal/api db:migrate    # creates all tables (migrations 001, 002)
pnpm --filter @reveal/api db:seed       # loads survey content + the 4 admin accounts
```

- **Migrate** builds the tables — student, report_instance, raw_capture, derived, report_payload, the instrument tables, and so on.
- **Seed** loads the live survey (version 1.0) and the admin team (`jahaanvi`, `prashant`, `reva`, `jayant`) with the temporary password `reveal@2026`. **Change these passwords after launch.**
- Both commands are **idempotent** — safe to re-run; they skip work already done.

Milestone 3 — Deploy the API server (Render)

1. In Render, choose **New** → **Blueprint** and pick this repo. Render detects `render.yaml` at the repo root (a "Blueprint"), which already declares:
   - build: `corepack enable` → install → build `@reveal/shared` → build `@reveal/api`
   - start: `pnpm --filter @reveal/api start`
   - a health check at `/health`
   - the free Postgres (`reveal-db`), with `DATABASE_URL` **auto-wired** into the API
   - `JWT_SECRET` **auto-generated** by Render (you don't invent one)
   - `SYNTHESIS_MODE=manual` (deterministic writer, no API key needed)
2. The only variable you must set by hand is `CORS_ORIGIN` — and you can only fill it once the website exists, so leave it for Milestone 5.
3. Deploy. When it's live you'll have an API URL like `https://reveal-api.onrender.com`. Confirm it by visiting `https://reveal-api.onrender.com/health` — it should return `{"status":"ok"}`.

> **Migrations:** the Blueprint creates the database but does **not** create the tables. Run Milestone 2 once from your laptop against the database's **External** connection string (Render dashboard → `reveal-db` → "External Database URL") before the API can serve anything.

Milestone 4 — Deploy the website (Vercel)

1. In Vercel, **import the repo**. Vercel picks up `vercel.json`, which already sets:
   - build: build `@reveal/shared` → build `@reveal/web`
   - output: `apps/web/dist`
   - SPA routing: every path rewrites to `index.html`
2. Set **one** environment variable: `VITE_API_URL` = your Render API URL from Milestone 3 (e.g. `https://reveal-api.onrender.com`). *(No trailing `/api` — the client appends that itself.)*
3. Deploy. You'll get a website URL like `https://reveal.vercel.app`.

Milestone 5 — Connect the two (the CORS handshake)

The API only accepts requests from origins you explicitly allow. Right now it doesn't know about your Vercel URL yet.

1. Go back to Render → the API service → environment.
2. Set `CORS_ORIGIN` to your Vercel URL (e.g. `https://reveal.vercel.app`). Multiple origins are comma-separated if you add a custom domain later.
3. Redeploy the API (or let Render redeploy on the env change).

> Symptom if you skip this: the site loads but every action fails with a CORS error in the browser console. Fixing `CORS_ORIGIN` resolves it.

Milestone 6 — Smoke-test the live product

Walk the real path end to end:

1. Open the Vercel site. **(First load may wake the sleeping API — give it a minute.)**
2. Sign up as a student → the dev verification code appears in the flow → verify.
3. Complete the survey and seal the session.
4. Sign in at `/admin/signin` as `jayant` / `reveal@2026` → find the pending report → **approve** it.
5. As the student, open the report → the full **Design Signature** renders.

If all five steps pass, you are live.

Milestone 7 — Go-live housekeeping

- [ ] **Change the admin passwords** away from `reveal@2026`.
- [ ] Confirm `JWT_SECRET` is a real long random string (the server refuses to boot in production on the insecure default — good).
- [ ] Decide on AI: leave `SYNTHESIS_MODE=manual` (deterministic writer, no key) or set `SYNTHESIS_MODE=auto` with an `ANTHROPIC_API_KEY` for AI prose.
- [ ] *(Optional)* Add a free uptime pinger on `/health` to soften cold starts.
- [ ] *(Optional)* Attach a custom domain in Vercel, and add it to `CORS_ORIGIN`.

Environment variables — the complete reference

Set these in your host's dashboard. **Never commit real secrets** — the repo's `.env` is git-ignored, and `.env.example` shows the shape.

Variable	Where it's set	Required?	What to set it to
`DATABASE_URL`	Render (API)	Auto	Auto-wired from `reveal-db` by the Blueprint (or set a Neon string for Route B)
`JWT_SECRET`	Render (API)	Auto	Auto-generated by the Blueprint (`generateValue`) — leave it
`CORS_ORIGIN`	Render (API)	**Yes**	Your Vercel site URL (comma-separated if several) — the one var you set by hand
`NODE_ENV`	Render (API)	Yes	`production` (already in `render.yaml`)
`PORT`	Render (API)	Auto	Render provides it; defaults to 4000 locally
`VITE_API_URL`	Vercel (web)	**Yes**	Your Render API URL (no trailing `/api`)
`SYNTHESIS_MODE`	Render (API)	No	`manual` (no AI, default-safe) or `auto` (live AI)
`ANTHROPIC_API_KEY`	Render (API)	Only if `auto`	Your Anthropic key
`SYNTHESIS_MODEL`	Render (API)	No	Defaults to `claude-sonnet-5`
`SYNTHESIS_TEMPERATURE`	Render (API)	No	Defaults to `0.3`
`JWT_EXPIRES_IN`	Render (API)	No	Defaults to `12h`
`STORAGE_BUCKET` / `STORAGE_*`	Render (API)	No	Only if wiring real file storage (e.g. Supabase Storage)

> **Which secrets are truly mandatory to go live:** `DATABASE_URL` , `JWT_SECRET` , `CORS_ORIGIN` (on the API) and `VITE_API_URL` (on the website). Everything else has a safe default.

---

8. When you outgrow the free tier

The free setup is genuinely production-shaped, but here's the upgrade order when the pilot succeeds:

Symptom of growth	First thing to upgrade	Rough cost
Cold starts annoy real users	Render API → paid "always-on" instance	~$7/mo

Symptom of growth	First thing to upgrade	Rough cost
Approaching the ~90-day Render DB clock	Move DB to **Neon** (or a paid Postgres)	$0 on Neon free, or ~$19/mo paid
Many portfolio/resume uploads	Real object storage (**Supabase Storage** / S3)	Free tier, then usage-based
Heavy report traffic	It already scales — static site on a CDN, cached reports	Usually still free
Live AI reports at volume	Anthropic API usage	Per-report, pay-as-you-go

Because reports are **generated once and cached**, and the website is **static files on a CDN**, REVEAL's cost curve stays remarkably flat as usage grows — the only piece that ever really needs paid upgrading first is the always-on API server.

---

9. One-page cheat sheet

```
1. DATABASE   → Neon: create project, copy connection string
2. SCHEMA     → locally: DATABASE_URL=… ; pnpm db:migrate ; pnpm db:seed
3. API        → Render: import repo (uses render.yaml)
                set DATABASE_URL, JWT_SECRET, CORS_ORIGIN
                verify /health returns ok
4. WEBSITE    → Vercel: import repo (uses vercel.json)
                set VITE_API_URL = the Render API URL
5. CONNECT    → Render: set CORS_ORIGIN = the Vercel site URL, redeploy
6. TEST       → sign up → survey → admin approve → read report
7. SECURE     → change admin passwords, confirm JWT_SECRET, pick SYNTHESIS_MODE

Total monthly cost: $0
```

Deploy configs referenced here already live in the repo: `render.yaml` *and* `vercel.json`*. Architecture background:* `ARCHITECTURE.md`*.*